

Boolean expressions

CSC103 Programming Fundamentals / Lecture 07

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

A ticket discount applies to ages below 12 or at least 60.
Write the condition and test ages 11, 12, 59 and 60.

Express each qualifying age group as
its own comparison.

Learning objectives

- Construct relational and logical conditions
- Evaluate Boolean expressions
- Choose explicit grouping for compound rules

CLO-2

Relational operators

Logical combinations

Intervals and precedence

Negation and boundary cases

Relational operators

- <, <=, > and >= compare numeric values.
- == tests primitive equality and != tests inequality.
- A relational expression produces true or false.

Example

```
int marks = 50;  
boolean passed = marks >= 50;  
boolean perfect = marks == 100;
```

Logical combinations

- `&&` requires both conditions to be true.
- `||` requires at least one condition to be true.
- `!` reverses a Boolean value.

Example

```
boolean eligible = age >= 18 && registered;  
boolean weekend = day == 6 || day == 7;
```

Intervals and precedence

- Write a range check as two comparisons joined with `&&`.
- Relational tests bind before `&&`, which binds before `||`.
- Parentheses make mixed rules easier to inspect.

Example

```
boolean valid = marks >= 0 && marks <= 100;  
boolean accepted = (a || b) && c;
```

Negation and boundary cases

- The opposite of $x < \text{limit}$ is $x \geq \text{limit}$.
- De Morgan: $!(a \ \&\& \ b)$ equals $!a \ || \ !b$.
- Tests at the boundary reveal missing equality cases.

Example

```
!(age >= 18 && registered)
```

Equivalent:

```
age < 18 || !registered
```

Translating a sentence into a condition

- "Below 12" uses < 12 , while "at least 60" uses ≥ 60 .
- The word "or" permits either qualifying group.
- Test the endpoints to confirm that the inequalities reflect the wording.

Example

```
Discount rule:  
age < 12 || age >= 60
```

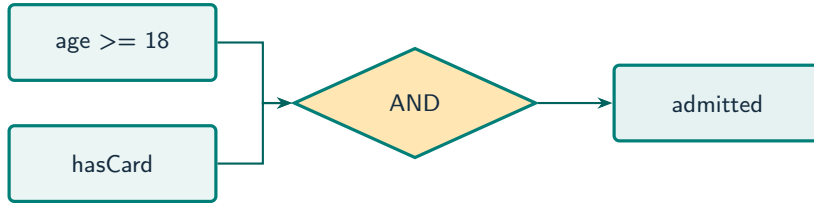
A Boolean expression is a value

- The result of a comparison can be stored, passed or combined.
- A condition must ultimately produce boolean in Java.
- A number such as 1 is not a replacement for true.

Example

```
boolean discounted = age < 12 || age >= 60;  
System.out.println(discounted);
```

Build a compound condition from two facts



What to notice

Admission requires both facts to be true. One true input is insufficient.

Key distinctions

A	B	A && B	A B
false	false	false	false
false	true	false	true
true	false	false	true
true	true	true	true

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
int marks = 50;  
boolean valid = marks >= 0 && marks <= 100;  
boolean passed = valid && marks >= 50;  
System.out.println(valid);  
System.out.println(passed);
```

First example: result

true	Intervals and precedence
true	Negation and boundary cases
50 satisfies the range and includes the passing boundary.	

Guided practice

A ticket discount applies to ages below 12 or at least 60.
Write the condition and test ages 11, 12, 59 and 60.

Attempt the solution before
continuing.
Use the definitions and examples
from this lecture.

Solution strategy

- Express each qualifying age group as its own comparison.
- Join the comparisons with `||`.
- Evaluate ages immediately below and at each boundary.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```
public class Solution07 {  
    public static void main(String[] args) throws Exception {  
        int age = 11;  
        System.out.println(age < 12 || age >= 60);  
        age = 12;  
        System.out.println(age < 12 || age >= 60);  
        age = 59;  
        System.out.println(age < 12 || age >= 60);  
        age = 60;  
        System.out.println(age < 12 || age >= 60);  
    }  
}
```

Execution trace

Age	age < 12	age >= 60	Discount
11	true	false	true
12	false	false	false
59	false	false	false
60	false	true	true

Follow the changing state in execution order.

Solution result

true
false
false
true

Why this result follows
Evaluate ages immediately below
and at each boundary.

A common incorrect approach

```
boolean inRange = 0 <= marks <= 100;
```

Example

Use `marks >= 0 && marks <= 100`. Each comparison must produce a Boolean operand `for` `&&`.

Boundary and failure checks

Check the stated input assumptions.

Write a library eligibility condition from two age and membership rules of your choice. Supply a truth table.

$\text{age} < 12 \parallel \text{age} \geq 60$

Results: true, false, false, true.

Check your understanding

Evaluate `true || false && false`. What is the opposite of `score >= 50`?

Write a prediction and explain the rule that supports it.

Answer and explanation

true, because `&&` binds first. The opposite is `score < 50`.

Use `marks >= 0 && marks <= 100`. Each comparison must produce a Boolean operand for `&&`.

Compare cases and predict outcomes

Age	Has card	<code>age >= 18 && hasCard</code>
17	true	false
18	false	false
18	true	true

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Admission requires age at least 18 and a valid card. Write both an admitted condition and its logical negation. Test the age boundary.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
boolean admitted = age >= 18 && hasCard;  
boolean denied = age < 18 || !hasCard;
```

- Negating AND changes it to OR and negates each operand.
- At age 18, the card determines the result.
- At age 17, admission is false even with a valid card.

Divisibility with AND, OR and XOR

- Remainder zero expresses divisibility by a nonzero integer.
- AND requires both properties; OR accepts either or both.
- Boolean XOR is true when exactly one property is true.

Source: [supplied ISB campus slides](#). See [speaker notes and ISB_Source_Map.md](#).

Worked problem: Divisibility with AND, OR and XOR

Task

For 6, evaluate divisibility by 2 and 3 using all three Boolean combinations.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture07 {  
    public static void main(String[] args) throws Exception {  
        int number = 6;  
        boolean by2 = number % 2 == 0;  
        boolean by3 = number % 3 == 0;  
        System.out.println(by2 && by3);  
        System.out.println(by2 || by3);  
        System.out.println(by2 ^ by3);  
    }  
}
```

Worked trace and expected result

Number	AND / OR	XOR
6	true / true	false
4	false / true	true
5	false / false	false

Expected console output

```
true  
true  
false
```

Extend the ISB example

Practice

Why would independent if statements be appropriate when displaying all properties that hold?

Explain your reasoning

More than one property can hold. An if-else chain would suppress later matching properties.

Lecture summary

- relational and logical conditions
- Boolean expressions
- explicit grouping for compound rules
- Express each qualifying age group as its own comparison.
- Join the comparisons with ||.
- Evaluate ages immediately below and at each boundary.

After-class practice

Write a library eligibility condition from two age and membership rules of your choice. Supply a truth table.

Syllabus reading

Deitel Ch 7 (as listed)

Next lecture: Selection with if and else